SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Dry Run Evaluation

Course Code: CSA17	**Course Title:** Artificial Intelligence
CO Assessed: CO1 – Apply (BL3)	**Assessment:** Assessment Tool 2 – Dry Run Evaluation
Weightage: 35% of CIA	**Duration / Marks:** 60 mins / 100 marks
Student Name: Deenesh Ra S T	**Reg. No.:** 192321149
Section / Batch: _____	**Date:** 11-10-2026

CO1 Statement: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Code of Conduct. I, Deenesh Ra S T (192321149), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Dry Run Evaluation: Answers

Every dry run shows the initial queue/stack/priority queue or belief state, each iteration, the assumptions used and the interpretation asked for. Conventions: graph search with an explored set, alphabetical expansion, goal test on removal from the frontier, ties broken alphabetically.

Question 1: Agents, Environments and Rationality – Dry Run of Agent Programs

(a) PEAS description [4 Marks]

Element	Description
Performance measure	Over T = 6 time steps: +10 for every square cleaned (a Suck on a Dirty square) and -1 for every Left or Right move. NoOp and a Suck on a clean square score 0.
Environment	Two squares A and B, each Clean or Dirty. The agent starts in A and both squares are initially Dirty; dirt never reappears.
Actuators	Suck, Left, Right, NoOp.
Sensors	A location sensor and a dirt sensor for the current square, giving the percept [location, status of own square].

(b) Classification of the task environment [4 Marks]

Property	Classification	Justification
Observability	Partially observable	The agent senses only its own square, not the other one.
Number of agents	Single agent	Only the vacuum agent acts; nobody else affects the squares.
Determinism	Deterministic	Every action has a certain outcome (Suck always cleans).
Episodic / Sequential	Sequential	Moving or sucking now changes what can be cleaned and scored later.
Static / Dynamic	Static	Nothing changes while the agent decides; dirt never reappears.
Discrete / Continuous	Discrete	Two squares, two statuses, four actions, six time steps.

(c) Dry run of X and Y for six steps [9 Marks]

Assumptions: Y's internal state starts empty and is updated after each step; it records a square as known clean once it has sucked it or sees it clean. After step 1 the state is {A} and after step 3 it is {A, B}.

Step	X: Percept	X: Action	Y: Percept	Y: Action	Y: Known-clean squares (after step)
1	[A, Dirty]	Suck	[A, Dirty]	Suck	{A}
2	[A, Clean]	Right	[A, Clean]	Right (B not known clean)	{A}
3	[B, Dirty]	Suck	[B, Dirty]	Suck	{A, B}
4	[B, Clean]	Left	[B, Clean]	NoOp (both known clean)	{A, B}
5	[A, Clean]	Right	[B, Clean]	NoOp	{A, B}
6	[B, Clean]	Left	[B, Clean]	NoOp	{A, B}

(d) Total performance [3 Marks]

Step	1	2	3	4	5	6	Total
X score	+10	-1	+10	-1	-1	-1	**16**
Y score	+10	-1	+10	0	0	0	**19**

X = 2 x 10 - 4 moves = 16; Y = 2 x 10 - 1 move = 19. Both clean the two squares (+20); X makes four moves (steps 2, 4, 5, 6) and Y only one (step 2).

(e) Interpretation [5 Marks]

Which agent is rational? Agent Y. Rationality means choosing, for every percept sequence, the action expected to maximise the performance measure given what the agent knows. Y scores 19, and an exhaustive check of all 4^6 action sequences shows 19 is the best possible score (cleaning both squares needs at least one move), so Y cannot be improved. X scores 16 because it keeps moving after the job is done, so it is not rational for this measure.

Why the same percept [B, Clean] at step 4 leads to different actions. X is a simple reflex agent: its program is condition-action rules on the *current percept only* ("if Clean, move to the other square"), so [B, Clean] always gives Left. Y is a model-based reflex agent: its action depends on the percept *and* an internal state built from the percept history. At step 4 its state is {A, B}, so it knows A was cleaned earlier and B is clean now, hence NoOp. The percept alone does not reveal the other square; only the remembered history does.

Rational versus omniscient. A rational agent maximises *expected* performance using its percepts and prior knowledge; it may be unlucky or err when information is missing. An omniscient agent knows the actual outcome of every action in advance, which is impossible. Y is rational without being omniscient: it never saw square B at the start; it simply used its percept history well.

Question 2: Foundations and History of AI; Problem-Solving Agent and Problem Formulation

Part A

(a) Classification of systems and ideas [8 Marks]

System / Idea	Approach to AI	Era / Milestone
Turing Test proposed (1950)	Acting humanly	Gestation of AI (1943-1955): Turing's 1950 paper "Computing Machinery and Intelligence".
General Problem Solver (Newell and Simon, 1957)	Thinking humanly (cognitive modelling: GPS was built to imitate human problem-solving steps)	Early enthusiasm, great expectations (1952-1969), following the 1956 Dartmouth workshop where the field was named.
Aristotle's syllogisms and formal logic	Thinking rationally ("laws of thought", the logicist tradition)	Philosophical foundations: ancient Greece (4th century BC), later basis of logic-based AI.
An agent that does the right thing to maximise expected performance	Acting rationally (the rational agent approach)	The intelligent-agent view, prominent from the mid-1990s to today.

Part B: Water-jug problem (X = 4 litres, Y = 3 litres)

(b) Problem formulation [6 Marks]

Component	Formulation
State representation	(x, y) with x = litres in jug X (0 to 4) and y = litres in jug Y (0 to 3), integers only; no measuring marks.
Initial state	(0, 0)
Actions	Fill X; Fill Y; Empty X; Empty Y; Pour X into Y; Pour Y into X (until the source is empty or the target is full).
Transition model	Fill X: (x, y) -> (4, y). Fill Y: (x, y) -> (x, 3). Empty X: (x, y) -> (0, y). Empty Y: (x, y) -> (x, 0). Pour X into Y: t = min(x, 3 - y), result (x - t, y + t). Pour Y into X: t = min(y, 4 - x), result (x + t, y - t).
Goal test	x = 2 (exactly 2 litres in jug X; y may be anything).
Path cost	1 per action, so the cost of a path is its number of actions.

(c) Successor states [3 Marks]

State	Action	Successor
(0, 0)	Fill X	(4, 0)
(0, 0)	Fill Y	(0, 3)
(4, 0)	Fill Y	(4, 3)
(4, 0)	Empty X	(0, 0)
(4, 0)	Pour X into Y (t = min(4, 3) = 3)	(1, 3)

Omitted as they leave the state unchanged: for (0, 0) Empty X, Empty Y and both pours; for (4, 0) Fill X, Empty Y and Pour Y into X.

(d) State space size and a shortest solution [3 Marks]

The state space has **5 x 4 = 20 states** (x = 0..4, y = 0..3). Only 14 of them are reachable from (0, 0); the 6 unreachable are (1,1), (1,2), (2,1), (2,2), (3,1), (3,2).

Step	Action	State
0	-	(0, 0)
1	Fill X	(4, 0)
2	Pour X into Y	(1, 3)
3	Empty Y	(1, 0)
4	Pour X into Y	(0, 1)
5	Fill X	(4, 1)
6	Pour X into Y	(2, 3) - goal (x = 2)

Path cost = 6. Breadth-first search shows no solution with fewer than 6 actions. The only other shortest solution is (0,0), (0,3), (3,0), (3,3), (4,2), (0,2), (2,0), also cost 6.

(e) Interpretation [5 Marks]

Phase	For the water-jug task
1. Goal formulation	Decide what to achieve: the set of states with exactly 2 litres in jug X. This limits the agent's objectives and says when it has succeeded.
2. Problem formulation	Choose an abstraction: states (x, y), the six fill/empty/pour actions, path cost 1 per action; ignore spilling, tap speed and similar details.
3. Search	Look for a sequence of actions from (0, 0) to a goal state, e.g. by breadth-first search (finds the 6-action solution above).
4. Execution	Carry out the six actions of the found solution one after another.

Why the solution can be executed without new percepts: the environment is fully observable, deterministic, discrete and known. Each action's outcome is certain, so the agent knows the state after every action and the plan cannot be surprised; it can ignore its percepts and run the plan "open loop". In a stochastic or partially observable environment it would have to monitor percepts and re-plan.

Question 3: Searching for Solutions – Uninformed Search Strategies

Graph: S-A 1, S-B 4, A-C 2, A-D 5, B-D 1, C-G 6, D-G 2. Adjacency (alphabetical): S: A, B | A: C, D, S | B: D, S | C: A, G | D: A, B, G | G: C, D. Start S, goal G.

(a) BFS (queue initially [S]) [6 Marks]

Iteration	Node dequeued	Queue after expansion	Explored set
1	S	[A, B]	{S}
2	A	[B, C, D]	{S, A}
3	B	[C, D] (D already queued, S explored: nothing added)	{S, A, B}
4	C	[D, G]	{S, A, B, C}
5	D	[G] (G already queued)	{S, A, B, C, D}
6	G (goal)	-	goal test succeeds, stop

Path from parents: G <- C <- A <- S, i.e. S-A-C-G.

(b) DFS (stack, top on the left) [5 Marks]

Iteration	Node popped	Stack after pushing children
1	S	[A, B]
2	A	[C, D, B] (S explored; C on top, popped first)
3	C	[G, D, B] (A explored)
4	G (goal)	stop; stack left [D, B]

Path from parents: G <- C <- A <- S, i.e. S-A-C-G.

(c) Uniform-cost search, entries Node:g(parent) [7 Marks]

Iteration	Node expanded	g(n)	Priority queue after expansion
1	S	0	A:1(S), B:4(S)
2	A	1	C:3(A), B:4(S), D:6(A)
3	C	3	B:4(S), D:6(A), G:9(C)
4	B	4	**D:5(B)**, G:9(C) (D replaced: 4 + 1 = 5 < 6)
5	D	5	**G:7(D)** (G replaced: 5 + 2 = 7 < 9)
6	G (goal)	7	- stop

Path from parents: G <- D <- B <- S, i.e. S-B-D-G.

(d) Comparison table [4 Marks]

Strategy	Path found	Path cost	Nodes removed from frontier	Optimal here?
BFS	S-A-C-G	1 + 2 + 6 = 9	6 (S, A, B, C, D, G)	No
DFS	S-A-C-G	1 + 2 + 6 = 9	4 (S, A, C, G)	No
UCS	S-B-D-G	4 + 1 + 2 = 7	6 (S, A, C, B, D, G)	Yes

(e) Interpretation [3 Marks]

UCS vs BFS. Both removed 6 nodes, but they order the frontier differently: BFS by depth (first in, first out), ignoring edge costs; UCS by the path cost g(n), and it replaces a frontier entry when a cheaper path is found. BFS reached G through C (S-A-C-G, 9). UCS first reached D at cost 6 via A, then found 5 via B (D:5(B)) and then G at 7 via D (G:7(D)). The equal node count is a coincidence, not a sign of equal quality: S-B-D-G has the same number of edges as S-A-C-G, but is cheaper.

DFS. It needed only 4 removals because it dived along the alphabetically first branch A, C and happened to reach G at once. That is luck of the child ordering, not merit: DFS ignores costs, so its path costs 9 against the optimal 7; it is not optimal in general, and without an explored set it can loop forever in cyclic graphs or follow an infinitely deep branch (incomplete). Its only advantage is small memory.

Explored set (BFS iteration 3). When B is dequeued its neighbours are D and S. D is already in the queue and S was expanded in iteration 1. Without the explored set (and the in-queue check) S and D would be added again, giving [C, D, D, S], and S would then re-generate A and B, and so on. In an undirected graph every edge leads back to a parent, so repeated states and cycles occur; the explored set ensures each state is expanded at most once.

Question 4: Search with Partial Information – Belief States

States: S1 (L,D,D), S2 (R,D,D), S3 (L,D,C), S4 (R,D,C), S5 (L,C,D), S6 (R,C,D), S7 (L,C,C), S8 (R,C,C). Goal states: S7 and S8 (both squares Clean).

(a) Belief-state formulation (sensorless) [3 Marks]

Component	Formulation
Initial belief state	{S1, S2, S3, S4, S5, S6, S7, S8}: the agent knows neither its location nor the dirt status.
Actions	Left, Right, Suck. Applying an action to a belief state gives the set of results of the action on every state in it.
Goal test	A belief state is a goal if **every** state in it is a goal state, i.e. it is a subset of {S7, S8}.
Path cost	1 per action (assumed), so the cost of a plan is its length.

(b) Plan [Right, Suck, Left, Suck] [6 Marks]

Step	Action	Belief state after action
0	(start)	{S1, S2, S3, S4, S5, S6, S7, S8}
1	Right	{S2, S4, S6, S8} (S1, S2 -> S2; S3, S4 -> S4; S5, S6 -> S6; S7, S8 -> S8)
2	Suck	{S4, S8} (S2 -> S4; S4 -> S4; S6 -> S8; S8 -> S8)
3	Left	{S3, S7} (S4 -> S3; S8 -> S7)
4	Suck	{S7} (S3 -> S7; S7 -> S7)

The final belief state {S7} is a subset of {S7, S8}, so the plan is a **solution**: it reaches the goal from every possible initial state, with a path cost of 4. A search over belief states confirms that no sensorless plan shorter than 4 actions exists.

(c) Percept-based update [8 Marks]

Method: each cycle has a *prediction* step (apply the action to every state in the belief state) and an *update* step (keep only the states consistent with the percept: same location and same status of the current square).

Step	Action	After action (predicted)	Percept	After percept (updated)
0	(start)	{S1, S2, S3, S4, S5, S6, S7, S8}	[Left, Dirty]	{S1, S3}
1	Suck	{S5, S7}	[Left, Clean]	{S5, S7}
2	Right	{S6, S8}	[Right, Dirty]	{S6}

Step 0: [Left, Dirty] keeps states with the agent on the left and the left square dirty: S1 and S3. Step 1: Suck gives S1 -> S5 and S3 -> S7; [Left, Clean] keeps both. Step 2: Right gives S5 -> S6 and S7 -> S8; [Right, Dirty] keeps only S6 because S8 has a clean right square. The agent now knows it is in S6 = (R, C, D), but S6 is not a goal state; it still has to Suck.

(d) Number of belief states [3 Marks]

A belief state is any subset of the 8 physical states, so there are 2^8 = **256** possible belief states (255 if the empty set is excluded). This makes belief-state search harder than search over physical states, since the number of nodes grows exponentially (2^N) with the number of physical states N instead of linearly (N), and each node is a set of states that is bigger to store and more expensive to update.

(e) Interpretation [5 Marks]

Why percepts shrink the belief state. A percept is evidence: states that could not have produced it are removed. In part (c) the first percept alone cuts 8 states to 2, and the third cuts {S6, S8} to {S6}, so the agent ends up with certainty about its state.
Difference from the sensorless case. In part (b) there are no percepts, so only the *actions* can reduce uncertainty. The sizes go 8, 4, 2, 2, 1 because different states are mapped to the same one (Right puts the agent on the right whatever its location, Suck cleans the square it is in). This is *coercion*: the plan forces the world into a known state. With sensors the agent can instead observe, update and choose its next action from what it has seen (a contingent plan), which is more flexible and usually shorter.
Why the sensorless plan is still rational. Rationality does not require knowing the state; it requires the best expected performance given the agent's percept sequence and prior knowledge. [Right, Suck, Left, Suck] succeeds from *all eight* possible starting states and is as short as any sensorless plan can be (4 actions), so no other plan the agent could choose without percepts would do better.

Rubrics for Calculation

Criteria	Weight (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation or unclear or missing reasoning.

Evaluation Page (for faculty use)

Student: Deenesh Ra S T Reg. No.: 192321149

Q. No. / Criteria	Understanding of Search Problem	State Expansion and Dry Run Execution	Application of Search Algorithm	Accuracy of Solution Path	Presentation and Logical Reasoning	Sub. Total	Total %
1							
2							
3							
4							
Total							

Faculty In-charge

Signature: _____
Date: _____

Course Coordinator

Signature: _____
Date: _____

Program Director

Signature: _____
Date: _____